

Required NuGet Packages

Install the following Entity Framework Core packages:

- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools
- Microsoft.EntityFrameworkCore.Proxies

Note: The packages must match the .NET Framework selected when creating the project.

Scenario: Hospital Equipment Maintenance System

A hospital needs a web-based system to manage its medical equipment and the maintenance performed on each item. The hospital currently has several pieces of equipment, such as patient monitors, ventilators, ECG machines, ultrasound machines, and infusion pumps. Each piece of equipment can have many maintenance records. The hospital wants staff to view equipment information and its associated maintenance history via an ASP.NET Core MVC application. You have been appointed to develop a prototype of the system.

The application must use Entity Framework Core and LINQ to retrieve data from SQL Server and must demonstrate three different approaches to loading related data:

- Eager Loading
 - Explicit Loading
 - Lazy Loading
-

Task 1 – Create the MVC Application and Database Model**[6 Marks]**

❖ Create an ASP.NET Core MVC application. **You must:**

Create an **Equipment model** containing appropriate properties for:

- Equipment ID
- Equipment Name
- Equipment Type
- Purchase Date

Create a **MaintenanceRecord model** containing appropriate properties for:

- Maintenance ID
- Maintenance Date
- Description
- Cost
- Equipment ID

Establish a one-to-many relationship where: One Equipment → Many Maintenance Records

- Create an Entity Framework Core DbContext.

- Store the SQL Server connection string in appsettings.json.
 - Configure the application to use SQL Server.
 - Use EnsureCreated() to create the database automatically.
-

Task 2 – Insert and Display Equipment Data

[4 Marks]

Populate the database with at least:

- 5 equipment records
- 10 maintenance records
- Each equipment item must have at least two maintenance records.
- Use suitable data based on the scenario.
- Create an MVC page that displays the equipment information in a table.

The page must display:

- Equipment ID
- Equipment Name
- Equipment Type
- Purchase Date

Verify the database and records using your database application / SQL Server.

Task 3 – LINQ Query

[4 Marks]

Create an MVC action that uses LINQ to retrieve all equipment belonging to the:

- Monitoring
- equipment type.

The results must be:

- filtered using LINQ;
- ordered alphabetically by equipment name; and
- displayed through an MVC Razor View.

The page must clearly display the retrieved equipment.

Task 4 – Eager Loading

[5 Marks]

- Create an MVC action and View that demonstrate Eager Loading.
- The application must retrieve a selected piece of equipment together with its related maintenance records.
- Use the appropriate Entity Framework Core method to load the related records as part of the database query.

Display:

- Equipment name
- Equipment type

- Purchase date
- Maintenance date
- Maintenance description
- Maintenance cost

Add a suitable comment in the controller explaining why the technique used represents Eager Loading.

Task 5 – Explicit Loading

[5 Marks]

- Create another MVC action and View demonstrating Explicit Loading.

The application must:

- Retrieve a selected equipment record.
- Load the equipment's maintenance records separately and explicitly.
- Display the equipment and its maintenance records.
- The solution must use Entity Framework Core's explicit loading functionality.
- Add a comment explaining how Explicit Loading differs from Eager Loading.

Task 6 – Lazy Loading

[4 Marks]

- Configure the MVC application to support Lazy Loading using Entity Framework Core proxies.

The application must:

- Configure the appropriate EF Core proxy package.
- Configure Lazy Loading in Program.cs.
- Configure the navigation properties appropriately.
- Retrieve a selected equipment item without using Eager Loading.
- Display its maintenance records through the navigation property.
- Add a comment explaining why the maintenance records are loaded automatically when the navigation property is accessed.

Deliverables:

Students must submit the complete Visual Studio project and demonstrate:

- The MVC application running successfully.
 - The database automatically created in SQL Server.
 - Equipment and maintenance data visible in the database application.
 - The LINQ query.
 - Eager Loading.
 - Explicit Loading.
 - Lazy Loading.
-

Sample Data for the Tables

1. Equipment Table

EquipmentId	EquipmentName	EquipmentType	PurchaseDate
1	Patient Monitor	Monitoring	2025-03-15
2	Ventilator	Life Support	2024-07-10
3	ECG Machine	Monitoring	2025-01-20
4	Ultrasound Machine	Imaging	2024-11-05
5	Infusion Pump	Medication	2025-06-18

2. MaintenanceRecord Table

MaintenanceId	MaintenanceDate	Description	Cost	EquipmentId
1	2026-01-15	Battery replaced	1500.00	1
2	2026-07-20	System inspection	800.00	1
3	2026-02-10	Annual service	2500.00	2
4	2026-08-05	Filter replaced	1200.00	2
5	2026-03-12	Electrical inspection	950.00	3
6	2026-08-18	Calibration completed	1100.00	3
7	2026-01-25	Probe inspection	1800.00	4
8	2026-06-30	Software update	750.00	4
9	2026-02-05	Pump calibration	650.00	5
10	2026-07-15	Battery replaced	500.00	5

Marking Rubric

Task 1 — MVC, Models, Relationship and Database Configuration

[6 Marks]

Criteria	Excellent	Satisfactory	Not Achieved	Marks
ASP.NET Core MVC project	Correctly creates and runs an ASP.NET Core MVC application targeting .NET xx	MVC application created but has minor configuration issues	Incorrect project type or application does not run	1
Equipment model	Correct model with ID, name, type and purchase date	Model created but one property/configuration is incorrect	Model missing or substantially incorrect	1
MaintenanceRecord model	Correct model with ID, date, description, cost and foreign key	Model created with minor errors	Model missing or substantially incorrect	1

Criteria	Excellent	Satisfactory	Not Achieved	Marks
One-to-many relationship	Correctly establishes Equipment → MaintenanceRecord relationship with navigation properties	Relationship exists but has minor issues	Relationship not implemented	1
DbContext and SQL Server	Correct DbContext, DbSet properties and SQL Server configuration	Mostly correct with minor errors	DbContext/database configuration missing or incorrect	1
appsettings.json and database creation	Connection string correctly stored in appsettings.json and EnsureCreated() successfully creates the database	Connection string/database creation partially correct	Connection string missing or database manually created	1

Task 2 — Sample Data and MVC Display

[4 Marks]

Criteria	Excellent	Satisfactory	Not Achieved	Marks
Equipment data	At least 5 suitable equipment records inserted correctly	3–4 records inserted	Fewer than 3 records or incorrect data	1
Maintenance data	At least 10 records inserted and correctly associated with equipment	6–9 records or some incorrect relationships	Fewer than 6 records	1
MVC View	Equipment data displayed clearly using a Razor View/table	Data displayed but presentation has minor issues	Data not displayed correctly	1
SSMS verification	Correctly demonstrates the automatically generated database and tables in SSMS 20	Database exists but verification is incomplete	Database/tables not successfully created	1

Task 3 — LINQ Query

[4 Marks]

Criteria	Excellent	Satisfactory	Not Achieved	Marks
LINQ filtering	Correctly uses LINQ to retrieve only Monitoring equipment	LINQ used but filtering is partially incorrect	No appropriate LINQ filtering	1
LINQ ordering	Correctly orders results by equipment name	Ordering attempted but incorrect	No ordering	1
MVC Controller/View	Correctly passes LINQ results from Controller to Razor View	Results partially passed/displayed	Results not displayed through MVC	1
Output	Clear and correct display of equipment information	Output contains minor errors	Output missing or incorrect	1

Task 4 — Eager Loading

[5 Marks]

Criteria	Excellent	Satisfactory	Not Achieved	Marks
Eager Loading implementation	Correctly uses Include() to load related maintenance records	Include() used but implementation has minor issues	No Eager Loading	1
Correct relationship	Correct equipment and its associated maintenance records are retrieved	Related records partially correct	Incorrect or unrelated records displayed	1
MVC implementation	Correct Controller action and Razor View	Minor MVC implementation errors	MVC implementation missing	1
Information displayed	Equipment and all required maintenance information displayed correctly	Some required information missing	Required information not displayed	1
Understanding/comment	Code comment clearly explains that related data is loaded with the original query	Basic explanation provided	No explanation	1

Task 5 — Explicit Loading

[5 Marks]

Criteria	Excellent	Satisfactory	Not Achieved	Marks
Initial equipment query	Retrieves equipment without initially requesting the collection	Query has minor issues	Incorrect query	1
Explicit Loading implementation	Correctly uses Entry().Collection().Load() to load maintenance records	Explicit Loading attempted but has minor errors	Explicit Loading not implemented	1
Correct related data	Correct maintenance records are displayed for the selected equipment	Some related data is incorrect/missing	Incorrect related records	1
MVC View	Correctly displays the equipment and maintenance records	Minor display problems	View missing or incorrect	1
Understanding/comment	Clearly explains that the programmer explicitly requests the related records	Basic explanation	No explanation	1

Task 6 — Lazy Loading

[4 Marks]

Criteria	Excellent	Satisfactory	Not Achieved	Marks
Lazy Loading configuration	Correctly configures UseLazyLoadingProxies() and required EF Core proxy package	Configuration partially correct	Lazy Loading not configured	1
Navigation properties	Correctly uses appropriate virtual navigation properties	Navigation property exists but configuration has minor issues	Navigation property not suitable for Lazy Loading	1
Lazy Loading implementation	Retrieves equipment without Include() and related records load when navigation property is accessed	Lazy Loading works but implementation is not fully demonstrated	No Lazy Loading	1
Understanding/comment	Clearly explains how Lazy Loading differs from Eager and Explicit Loading	Basic explanation	No explanation	1